



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Невен Илинчић

Имплементација игре Commander's Defense применом клијент-сервер архитектуре

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Невен Илинчић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Имплементација игре Commander's Defense применом клијент-сервер архитектуре
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	6/45/10/0/42/0/0
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Neven Ilinčić
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Implementation of the game Commander's Defense using client-server architecture
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	6/45/10/0/42/0/0
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	<i>Online multiplayer</i> систем	3
2.1	<i>Online Multiplayer</i> игре	3
2.2	Архитектуре <i>online multiplayer</i> система	3
3	Концепти на којима се заснива клијент-сервер архитектура	7
3.1	Серверско усклађивање	7
3.2	<i>Command</i> шаблон	7
3.3	Клијентска предикција	7
3.4	<i>Tick-rate</i> синхронизација и детерминизам	7
4	<i>Commander's Defense</i>	9
4.1	О игри	9
4.2	Приказ игре	10
4.3	Покренута партија	13
5	Имплементација	17
5.1	Коришћене технологије	17
5.2	Синхронизација стања	18
5.3	Имплементација кретања играча - <i>Command</i> и <i>State</i> шаблон	22
5.4	Комуникација клијента и сервера	27
6	Закључак	35
	Списак слика	37
	Списак листинга	39
	Биографија	41
	Литература	43
	Грешке у литературним наводима	45

Развој видео игара током последњих деценија довео је до појаве *online multiplayer* игара које представљају један од најзначајнијих грана савремене индустрије. Њихов развој је у великој мери условљен напретком рачунарских система и интернета, као и појавом платформи за дистрибуцију и стримовање садржаја као што су *YouTube* и *Twitch*. Поред тога, појава е-спорта допринела је значајном порасту популарности овог типа игара и њиховом позиционирању као важног сегмента индустрије видео игара [1].

За разлику од једнокорисничких (енгл. *singleplayer*) игара, *online multiplayer* системи захтевају континуирану размену података између више учесника у реалном времену (енгл. *real-time*), што доводи до повећане сложености у њиховом дизајну и имплементацији. Због тога је неопходно обезбедити стабилну мрежну комуникацију и конзистентно стање игре код свих клијената, што представља један од кључних изазова у развоју оваквих система. Такође, *online multiplayer* игре су подложне различитим облицима злоупотребе, при чему корисници могу покушати да на непрописан начин стекну предност у односу на друге, што додатно повећава сложеност њихове имплементације. Овај проблем је посебно изражен у компетитивним играма које се заснивају на принципу рангирања и добијања награда, где је очување интегритета од изузетног значаја.

У оквиру овог рада је имплементирана рана верзија *online multiplayer* игре *Commander's Defense*, која се заснива на клијент-сервер архитектури. Систем је пројектован тако да сервер има централну улогу у обради логике игре и синхронизацији стања, док клијент служи за приказ и интеракцију са корисником. Игра је инспирисана постојећим насловима као што су *Commando Assault* и *Strike Force Heroes*, који су послужили као основа за идејни концепт и жанровски правац. Имплементација је извршена у циљу приказа практичне примене клијент-сервер архитектуре у развоју *real-time online multiplayer* игара.

Циљ рада је анализа и имплементација *online multiplayer* система заснованог на клијент-сервер архитектури, као и приказ кључних концепата који омогућавају његово стабилно функционисање у условима мрежне комуникације.

У наставку рада биће приказане теоријске основе *online multiplayer* игара, архитектонски приступи, као и детаљан опис дизајна и имплементације развијеног система.

2.1 Online Multiplayer игре

Online multiplayer игре представљају врсту видео игара које омогућавају истовремену интеракцију већег броја корисника путем рачунарских мрежа. За разлику од једнокорисничких игара, где је целокупна логика извршавања локализована на једном уређају, *online multiplayer* игре подразумевају размену података између више учесника у реалном времену. Ова размена података обухвата информације о позицији играча, акцијама, стању окружења и другим елементима који су неопходни за правилно функционисање игре.

Приликом развоја оваквих игара, потребно је водити рачуна о следећим карактеристикама:

- **Управљање мрежним кашњењем**
- **Скалабилност**
- **Робусност**
- **Конзистентност**
- **Отпорност на варање**

2.2 Архитектуре online multiplayer система

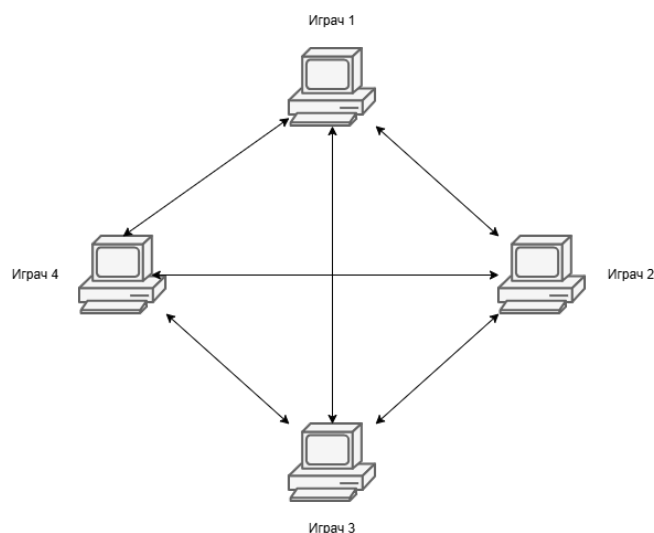
Приликом креирања *online multiplayer* игре, могуће је изабрати између *Peer-to-Peer*, клијент-сервер и хибридне архитектуре [2]. Одабир жељене архитектуре зависи од врсте игре, степена жељене заштите против варања и циљне групе играча. Свака од претходно наведених архитектура захтева специфичан приступ имплементацији како би се осигурала стабилност и синхронизација података између учесника [2].

2.2.1 Peer-to-Peer (P2P)

Peer-to-Peer представља дистрибуирани тип архитектуре код којег не постоји централизован систем, већ се размена података обавља директном комуникацијом између свих учесника, што је приказано на слици 1. У овом моделу, сваки учесник (чвор) има двоструку улогу: делује као клијент и сервер, обрађујући сопствене улазе и шаљући их осталим учесницима у мрежи.

Карактеристике архитектуре:

- Дистрибуирана контрола - сваки учесник је одговоран за израчунавање стања игре на локалном уређају. Ово захтева висок степен детерминизма у извршавању логике како би се осигурало да сви играчи, на основу истих улазних података, генеришу идентичан приказ стања света.
- Скалабилност и ресурси - будући да не постоји централни сервер који обрађује све податке, оптерећење се равномерно распоређује на све учеснике. Са повећањем броја играча повећавају се и укупни ресурси мреже (процесорска снага и пропусни опсег), али се истовремено повећава и број међусобних конекција, што може довести до мрежног zasiћења.
- Ниско мрежно кашњење - како се размена података врши директном комуникацијом између два играча, избегава се додатно кашњење које би увео посредник у виду централног сервера.
- Слаба отпорност на варање - главни проблем и уједино најслабија тачка P2P архитектуре. Одсуство централног ауторитета који би вршио валидацију података омогућава малициозним корисницима да манипулишу сопственим стањем (нпр. позицијом или ресурсима) пре слања осталим учесницима.
- Робусност и стабилност - систем је осетљив на стабилност везе сваког појединачног учесника. Уколико један корисник искуси висок проценат губитка пакета или спору интернет везу, то директно може изазвати десинхронизацију или застој игре код свих осталих учесника у мрежи.



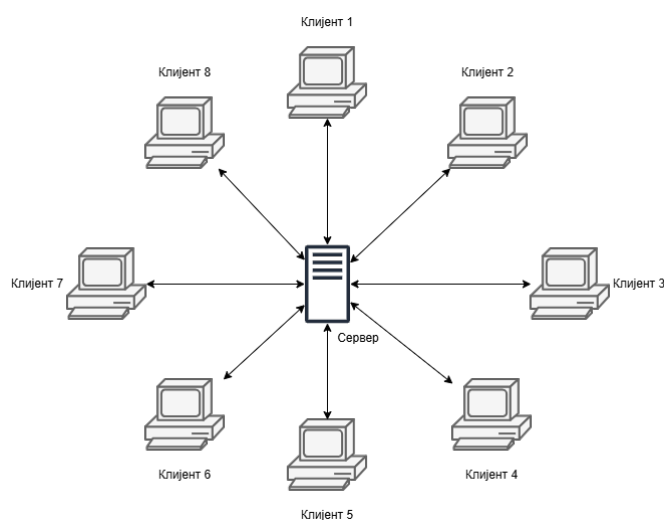
Слика 1: *Peer-to-Peer* архитектура.

2.2.2 Клијент-сервер архитектура

Клијент-сервер архитектура представља централизовани модел у којем су сви учесници (клијенти) повезани на један централни чвор - сервер. За разлику од P2P модела, клијенти не комуницирају директно међусобно, већ се свака размена података одвија искључиво преко сервера (приказан на слици 2).

Карактеристике архитектуре:

- Централизован ауторитет - сервер управља комплетном логиком игре. Клијенти шаљу само своје „намере“, док сервер валидира те податке и свима шаље потврђено стање.
- Висок степен отпорности на варање - како се сви критични прорачуни врше на серверу, локална манипулација меморијом (нпр. мењање количине здравља или брзине играча) не утиче на стварно стање игре. Сервер одбацује све нерегуларне податке које клијенти шаљу.
- Гарантована конзистентност - сви играчи добијају ажурирања са истог места. Тиме се постиже глобална конзистентност, јер сви учесници виде исто стање игре у сваком тренутку.
- Скалабилност и једна тачка отказа - главни проблеми клијент-сервер архитектуре. Како би сервер обрадио податке свих учесника, неопходно је да поседује значајан пропусни опсег и процесорску снагу ако треба да обради податке већег броја клијената. Такође, уколико дође до пада сервера, игра се прекида за све повезане учеснике.



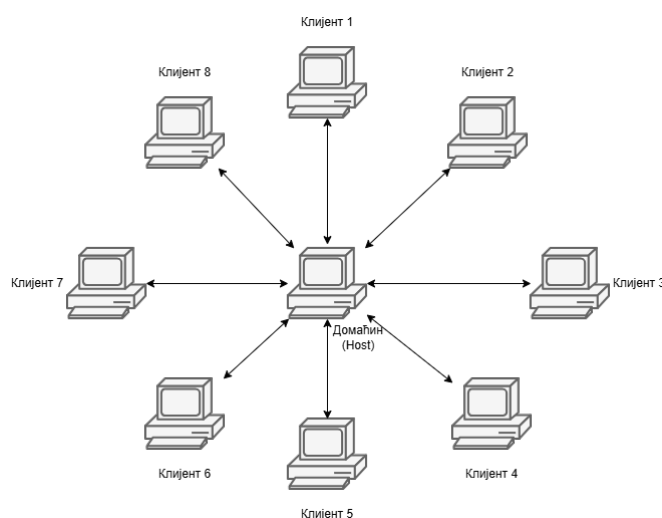
Слика 2: клијент-сервер модел.

2.2.3 Хибридни модел

Хибридни модел представља комбинацију P2P и клијент-сервер архитектуре покушавајући да искористи предности обе како би се постигао баланс између цене, перформанси и безбедности. У овом моделу, одређене функције су централизоване, док се друге обављају директно између играча. У архитектури видео игара, овај модел се најчешће појављује у облику где један играч преузима улогу сервера (*Host*) истовремено понашајући се као клијент (приказан на слици 3).

Карактеристике архитектуре:

- Централизована мета-логика - углавном се користи „мањи“ централизовани сервер који је одговоран за повезивање учесника.
- Скалабилност - како се број играча повећава, број *host*-ова аутоматски расте.
- Смањење мрежног кашњења (за домаћина) - играч који је изабран за домаћина има нулто кашњење, док остали играчи, ако се налазе географски близу њега, могу имати такође умањено мрежно кашњење.
- Миграција домаћина (енгл. *host migration*) - главни проблем архитектуре. Ако домаћин напусти партију, или изгуби конекцију, цела партија се прекида док се не изабере нови домаћин и не пренесе тренутно стање игре на његов рачунар.
- Слаба безбедност и предност домаћина - како домаћин има нулто кашњење, он је први који види измену стања игре, што му даје предност у односу на друге играче. Такође, домаћин представља „ауторитет“ за ту партију и може лако да мења податке у своју корист пре него што их пошаље осталим корисницима.
- Зависност од конекције домаћина - ако домаћин има слабију мрежну конекцију, остали учесници ће осетити кашњење (енгл. *lag*) без обзира на њихов квалитет конекције.



Слика 3: хибридни модел.

2.2.4 Избор архитектуре

За имплементацију игре *Commander's Defense* одабрана је клијент-сервер архитектура са наменским ауторитативним сервером. Иако овај модел захтева највеће ресурсе на страни инфраструктуре, он је једини који у потпуности задовољава строге захтеве за:

1. **Спречавањем варања** - онемогућава се манипулација подацима на страни клијената.
2. **Глобалном конзистентношћу** - сервер гарантује да сви учесници виде идентично стање игре у сваком тренутку, што је кључно за карактер игре.

Глава 3

Концепти на којима се заснива клијент-сервер архитектура

Пре приказа имплементације игре *Commander's Defense* потребно је објаснити концепте на којима се заснива клијент-сервер архитектура у видео играма. Имплементација ових концепата је кључна за одржавање флуидности игре и спречавања варања, што је неопходно како би се остварило позитивно корисничко искуство.

Битни концепти су:

- серверско усклађивање (енгл. *server reconciliation*)
- клијентска предикција (енгл. *client-side prediction*)
- примена *Command* шаблона
- *tick-rate* синхронизација

3.1 Серверско усклађивање

Серверско усклађивање представља механизам којим сервер исправља стање клијента у случају десинхронизације. Механизам се активира када год дође до одступања између предвиђеног стања на клијенту и званичног стања света на серверу [3]. Како би серверско усклађивање могло да се примени, неопходно је да клијент чува историју свих послатих команди у периоду између добијања два пакета са сервера,. У случају десинхронизације, сачуване команде се извршавају истим редоследом којим су и сачуване. Механизам је кључан за спречавање варања, јер сервер присилно враћа клијента у валидно стање.

3.2 *Command* шаблон

Како би серверско усклађивање могло да се примени, корисничке акције се енкапсулирају у објекте. Свака команда садржи секвенцијални број, што омогућава клијенту да понови акције у правом редоследу. Такође, секвенцијални број омогућава серверу да идентификује застареле команде и спречи напад репродукцијом (енгл. *replay attack*) код којег би нападач понављањем већ послатих пакета могао вишеструко извршити исту акцију [4].

3.3 Клијентска предикција

Техника омогућава клијенту да тренутно прикаже резултат корисничког уноса, пре него што сервер потврди акцију [3]. Тиме се визуелно елиминише кашњење које би иначе настало чекањем одговора сервера [3]. Уколико се предикција не би користила, играч би осетио застој између притиска тастера или помераја миша и акција на екрану, што нарушава корисничко искуство.

3.4 *Tick-rate* синхронизација и детерминизам

Tick-rate представља учесталост којом сервер обрађује логику и физику игре [3]. Фиксни интервал ажурирања омогућава детерминистичко понашање система, чиме се обезбеђује да ће исти улазни подаци довести до истог стања игре. Без *tick-rate* синхронизације између клијента и сервера, дошло би до сталних грешака у предикцији клијента и честог активирања серверског усклађивања.

Глава 4

Commander's Defense

4.1 О игри

Commander's Defense је 2D online multiplayer пуцачина са бочним скроловањем (енгл. *side-scroll shooter*). Игра садржи два режима игре (енгл. *game mode*):

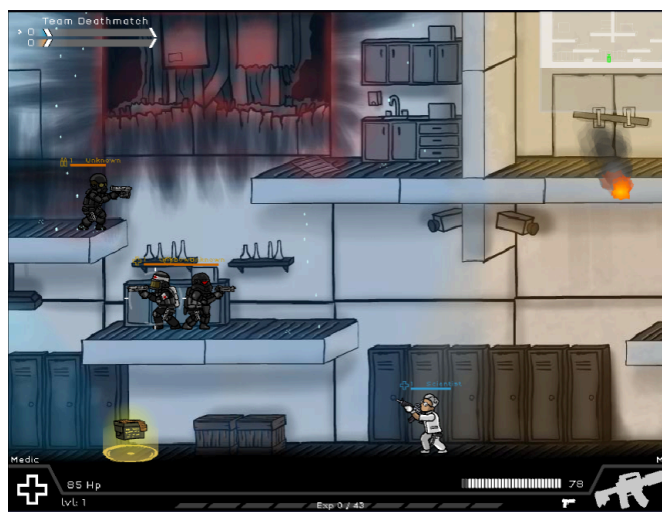
1. **Tower/Hangar** режим - циљ је уништити непријатељску кулу (хангар). Штета кули може да се наноси само док је непријатељски играч елиминисан. Овај режим је дизајниран за два играча.
2. Режим „свако против свакога“ (енгл. **Free For All - FFA**) - играчи се боре једни против других. Први играч који достигне постављену количину поена побеђује. Режим је дизајниран за два до десет играча.

Сваки играч у холу (енгл. *lobby*), пре почетка партије, има могућност одабира између три изгледа: командант у зеленом, плавом или црвеном оделу. Такође, сваки играч поседује три врсте оружја: пиштољ, аутоматску пушку и једну гранату.

Како је речено у уводу, игра је инспирисана двема играма. Режим куле је инспирисан игром *Commando Assault* (видети слику 4) док је FFA инспирисан игром *Strike Force Heroes* (видети слику 5).



Слика 4: *Commando Assault* видео игра.



Слика 5: *Strike Force Heroes* видео игра.

4.2 Приказ игре

4.2.1 Почетни мени

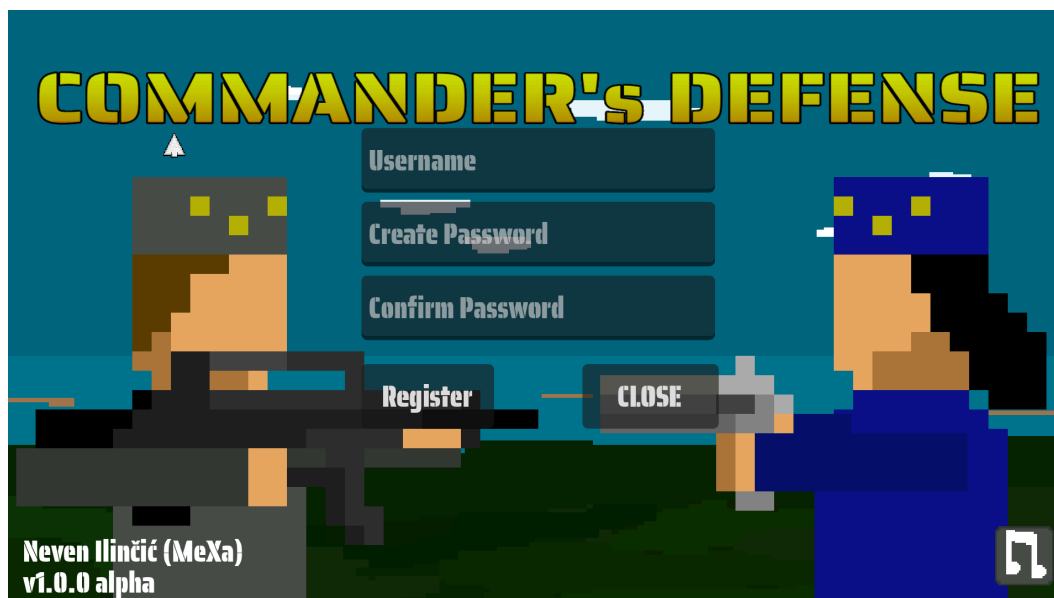
Након учитавања апликације, кориснику се приказује почетни мени (приказано на слици 6).



Слика 6: Почетни мени.

4.2.2 Мени за регистрацију

Пре него што корисник може да приступи игри, неопходно је прво да се региструје. Приликом регистрације, потребно је одабрати корисничко име и лозинку (приказано на слици 7).



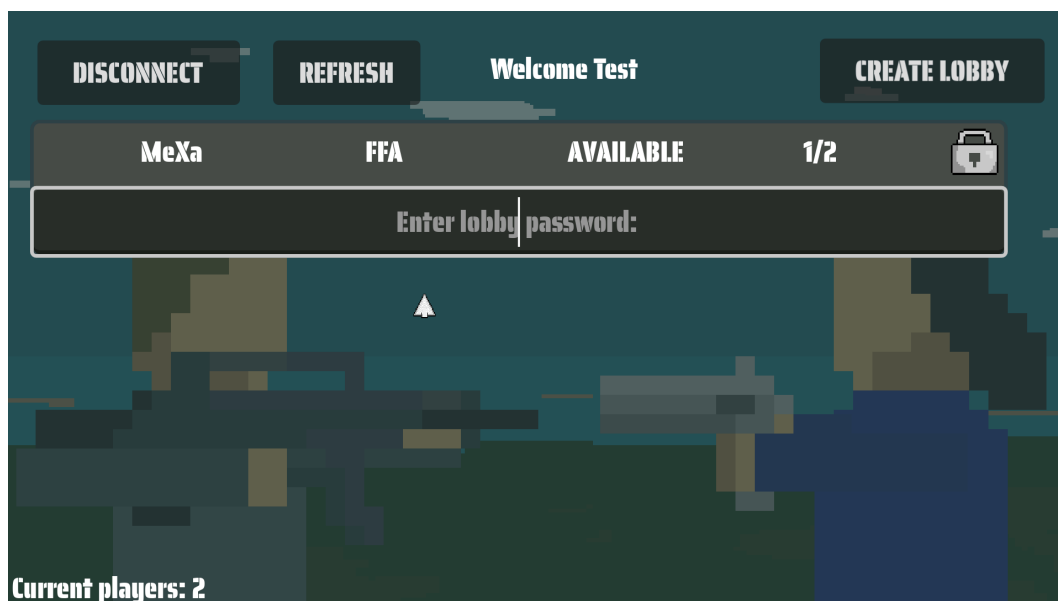
Слика 7: Мени за регистрацију.

4.2.3 Главни мени

Након успешне регистрације и пријављивања, кориснику се приказује главни мени игре у којем корисник има опцију да креира нови *lobby* или да се придружи већ постојећем (приказан на слици 8). Такође, за сваки *lobby* се налазе основне информације о њему:

1. Ко је домаћин партије (било да је креирао *lobby* или му је додељен, због одласка претходног домаћина).
2. За који режим игре је *lobby* креиран.

3. Стање *lobby*-ја - да ли је партија започета или не.
4. Тренутна попуњеност као и максималан капацитет *lobby*-ја.
5. Иконица која приказује да ли је *lobby* закључан. Ако јесте, како би му се приступило, неопходно је унети одговарајућу лозинку коју је поставио домаћин приликом његовог креирања.



Слика 8: Главни мени.

4.2.4 Мени за креирање *lobby*-ја

У случају да корисник одабере опцију да креира сопствени *lobby*, приказаће му се мени за креирање (видети слику 9). Кориснику се нуде три опције:

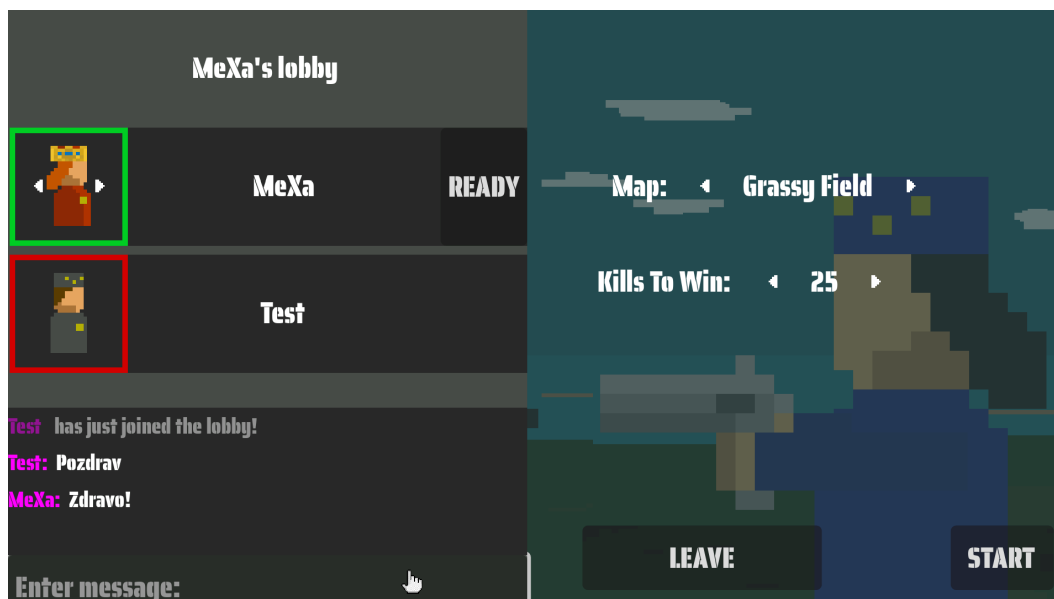
1. Падајући мени за избор режима игре - режим куле или FFA.
2. Падајући мени за избор максималног капацитета *lobby*-ја. У случају да је одабран режим куле, једина могућа вредност за одабир је два, док у случају FFA може да бира од два до десет.
3. Опциони параметар за постављање лозинке, у случају да корисник не жели да било ко може да приступи.

Слика 9: Мени за креирање *lobby*-ја.

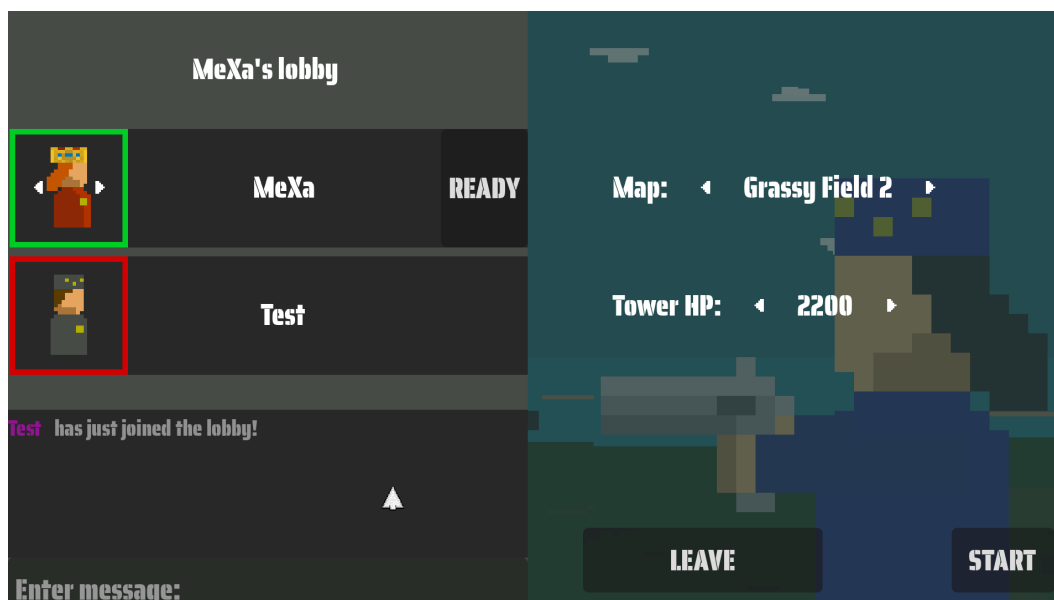
4.2.5 Креирани lobby

Након креирања или одабира lobby-ја, корисник приступа менију lobby-ја (приказан на слици 10). Корисник може да:

1. мења изглед играча притиском белих стрелица,
2. означи да је спреман за почетак партије притиском на дугме *Ready*,
3. види све поруке које су послате након што је корисник приступио lobby-ју и пошаље сопствену поруку.
4. У случају да је домаћин, корисник има могућност да одабере мапу и подеси неопходан број елиминација (за FFA режим) или подеси колико здравствених поена (енгл. *health points*) има кула (приказано на слици 11) и
5. покрене партију притиском *Start* дугмета уколико су сви играчи спремни.



Слика 10: Пример креираног lobby-ја - FFA режим.



Слика 11: Пример креираног lobby-ја - кула режим.

4.3 Покренута партија

4.3.1 UI

Притиском *Start* дугмета, домаћин започиње партију и сви играчи се додају на одабрану мапу. Слика 12 приказује изглед UI-ја када је игра покренута преко рачунара, а слика 13 када је покренута преко мобилног телефона.



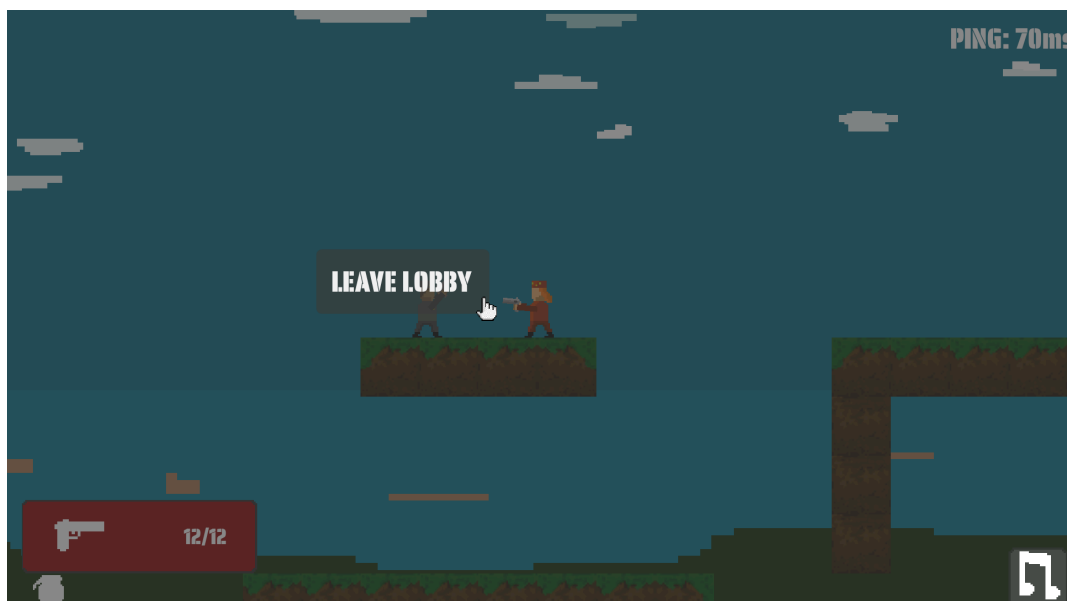
Слика 12: Изглед UI-ја игре покренуте на рачунару.



Слика 13: Изглед UI-ја игре покренуте на мобилном телефону.

4.3.2 Паузирајући мени

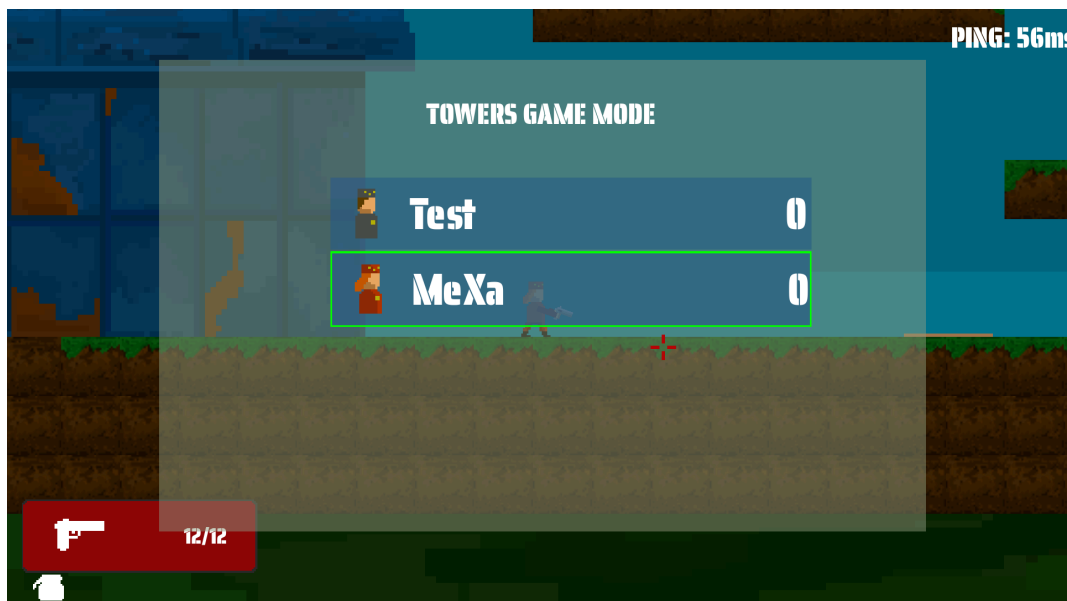
У току партије, корисник има могућност отварања паузирајућег менија (слика 14). Одатле, корисник може да изађе из покренуте партије или да пригуши или укључи звук позадинске музике.



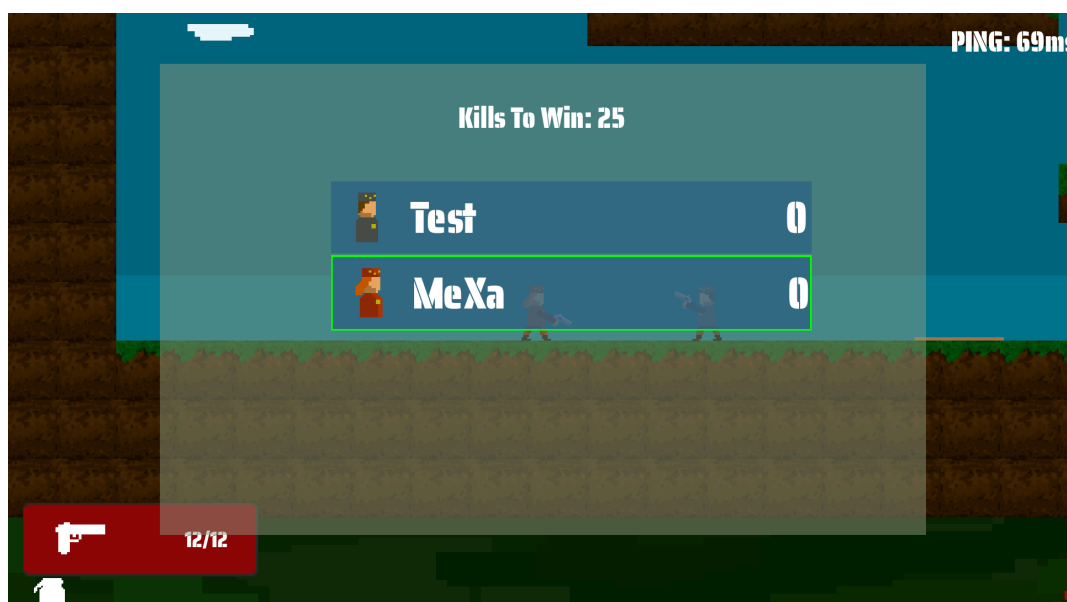
Слика 14: Изглед паузирајућег менија.

4.3.3 Scoreboard

Корисник у сваком тренутку може да види колико поена је остварио, као и поредак на табели (видети слику 15). Поред поена, корисник може да види који све играчи су присутни у партији, као и како сваки играч изгледа. Када се игра FFA режим, на врху табеле се исписује колико је потребно остварити поена како би се партија завршила, што је приказано на слици 16.



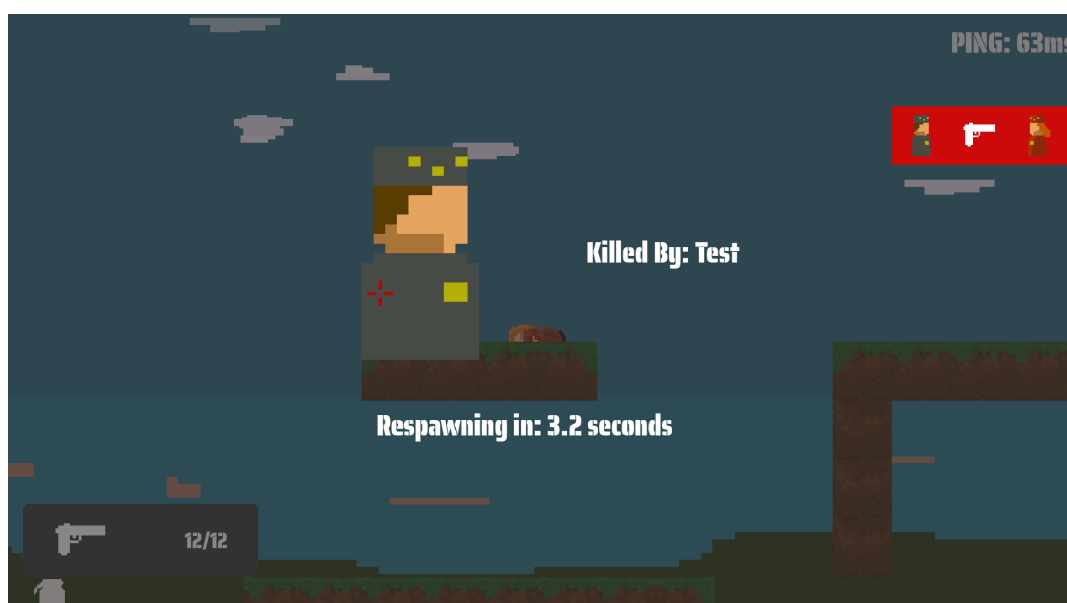
Слика 15: Изглед scoreboard-а са два играча у партији - FFA режим.



Слика 16: Изглед scoreboard-a са два играча у партији - FFA режим.

4.3.4 Death екран

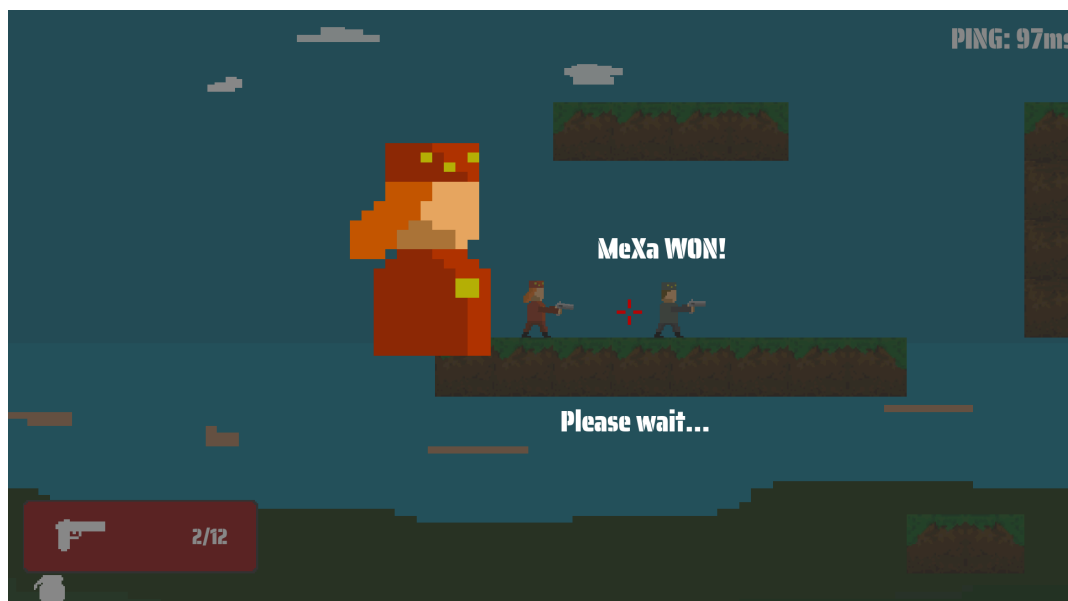
Ако је корисник елиминисан од стране другог играча, биће му приказан сиви екран (видети слику 17). На екрану је могуће видети корисничко име играча који га је елиминисао, његов изглед и преостало време док играч не оживи. Такође, могуће је видети помоћу којег оружја је играч елиминисан.



Слика 17: Изглед scoreboard-a са два играча у партији.

4.3.5 Екран завршене партије

Када се партија заврши, свим корисницима се приказује екран завршене партије (видети слику 18). На њему се може видети изглед и корисничко име играча који је остварио победу у партији.



Слика 18: Изглед *scoreboard*-а са два играча у партији.

По завршетку партије, сви играчи се пребацују назад у *lobby*.

5.1 Коришћене технологије

Клијентска страна игре *Commander's Defense* је имплементирана употребном *Godot Game Engine*-а, док је серверска страна имплементирана употребом програмског језика *Rust*. За базу података је коришћен PostgreSQL.

5.1.1 Rust

Како ауторитативни сервер представља центар система који истовремено комуницира са свим учесницима, потребно је обезбедити високе перформансе и осигурати безбедност меморије. Одсуство сакупљача смећа (енгл. *garbage collector*) у програмском језику *Rust* омогућава високе перформансе чинећи га погодним за имплементацију серверске стране [5]. Непостојање додатних трошкова као што су праћење и периодично ослобађање меморије значи да не долази до наглих скокова у заузећу процесора и микро-застоја у извршавању физичке симулације [5].

Поред меморијске ефикасности, *Rust* пружа и сигурност при раду са више нити. С обзиром да сервер истовремено мора да прима надолазеће пакете од већег броја клијената и паралелно ажурира стања партија, строга правила језика онемогућавају појаву грешака у управљању меморијом [5]. Немогућност појаве трке података (енгл. *data race*), прекомерног читања бафера (енгл. *buffer overread*) или прекомерног писања у бафер (енгл. *buffer overflow*) осигуравају да сервер буде заштићен од непредвидивих грешака које малициозни корисници могу искористити како би стекли предност у односу на друге кориснике [5].

5.1.2 Godot Game Engine

Иако је релативно нов и константно у развоју, *Godot Game Engine* је изузетно погодан за развој 2D игара. Овај погон се заснива на чворовима (енгл. *nodes*) и сценама, што омогућава модуларну и флексибилну организацију игре. Садржи развојно окружење, чиме се елиминише потреба за покретањем додатних спољних програма. Поред сопственог језика *GDScript*, чија је синтакса слична програмском језику *Python*, систем такође пружа подршку за развој у језику *C#*. Основни начин комуникације између различитих чворова је путем сигнала или *autoload* скрипти.

5.1.3 PostgreSQL

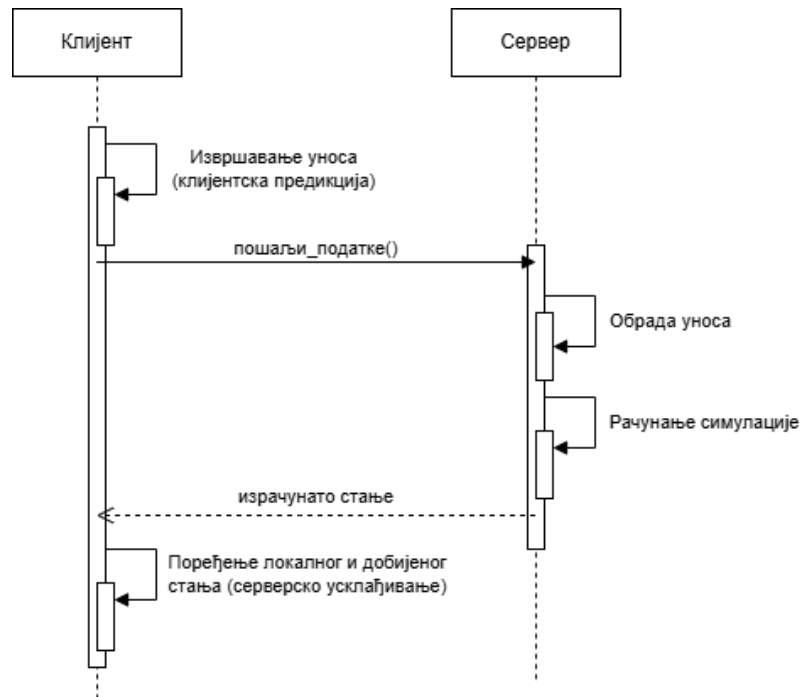
Како би се осигурало да не могу постојати два играча са истим називом, база података се састоји од једне табеле *players* са три колоне: *id*, *nickname* и *password*. Пример изгледа табеле се може видети на слици 19.

	id [PK] integer	nickname text	password text
1	1	Test	\$2b\$12\$AMhHyIbOrxGRl3t7J5EuKeHqNKsCzXeD0un9IbpC1TkEanxec...
2	2	Test2	\$2b\$12\$syaJZreUSMuOeySCvqDcZuOXdFSgSJDpgu7nbiabLmngURC7...
3	3	MeXa	\$2b\$12\$skzc5vM7ZwB11ZCpTICVAP099JEe1eFlxduVIW06V65bAW7IX...
4	5	Test3	\$2b\$12\$Ytt9BJ86Lx7t31K/TYCN.uLgFArJfBHaMtPajtmRB0LQDAVxYZ...

Слика 19: Изглед табеле *players* у бази података.

5.2 Синхронизација стања

Процес синхронизације стања представља критичну компоненту мрежне архитектуре, коју визуелно представља слика 20.



Слика 20: Дијаграм секвенци који приказује ток синхронизације стања.

Као што је приказано на дијаграму, клијент не чека потврду сервера како би ажурирао позицију, већ врши локалну предикцију. У тренутку када прими ауторитативно стање, врши се поређење које може довести до *rollback* процеса, чиме се клијент враћа у исправно стање израчунато од стране сервера.

5.2.1 Извршавање и чување акција (клијент)

Приказани код у листингу 1 демонстрира имплементацију клијентске предикције. Кључни део ове имплементације је ажурирање локалне позиције играча (***apply_movement_step*** метода - листинг 2) и чување у структуру ***state_history***. Тиме се обезбеђује референца за будући *rollback* процес. Детаљније о имплементацији клијентског кретања у секцији 5.3.

У листингу 3 приказана је ***send_data*** метода која се бави серијализацијом података о уносу.

```
func _physics_process(delta: float) -> void:
    var move_command: PlayerMoveCommand = handle_move_inputs(Network.INPUT_DATA["input_id"],
delta) #Енкапсулација корисничког уноса
    apply_movement_step(move_command, PHYSICS_DELTA) #Извршавање акције

    if not self.message_input.visible and not self.pause_menu.visible:
        Network.INPUT_DATA["input_id"] += 1
        self.player_state.handle_inputs(self)
        #Чување позиције играча у тренутку слања команде
        state_history.append(
            {
                "id": Network.INPUT_DATA["input_id"],
                "global_position": global_position
            }
        )
        send_data(move_command) #Слање команде ка серверу
```

Листинг 1: Имплементација позива

```
func apply_movement_step(command: PlayerMoveCommand, delta: float):  
    command.execute(delta)
```

Листинг 2: *apply_movement_step* метода

```
func send_data(move_command: PlayerMoveCommand):  
    if !Network.is_disconnecting:  
        var packed_byte_array: PackedByteArray = Network.convert_input_data_to_byte_array()  
        Network.send_data(packed_byte_array)  
        inputs_list.append(move_command)  
        if inputs_list.size() > 30:  
            inputs_list = inputs_list.slice(-30)
```

Листинг 3: Слање података ка серверу и чување команди у бафер.

Бафер *inputs_list* памти историју акција које су извршене, а које још увек нису потврђене од стране сервера.

5.2.2 Серверска обрада и валидација

Док клијент врши предикцију, сервер делује као ауторитативни ентитет који прима команде, валидира их и шаље коначно стање назад. Ово спречава варање и обезбеђује конзистентност игре за све играче. На листингу 4 се може видети део методе која је задужена за валидацију и обраду добијених пакета од клијената.

```
pub async fn handle_client_input(&
mut self, input: ClientInput, ip_address: SocketAddr) {
    self.address_to_players.insert(input.player_id, ip_address);
    match input.command {
        CommandEnum:
            :UdpPunch => {
                return;
            }
        _ => {}
    }
    if let Some(player) = self.players.get_mut(&input.player_id) {
        player.last_seen = Instant::now();
        //Валидација уноса
        if player.last_processed_input_id >= input.input_id {
            return;
        }
        player.last_processed_input_id = input.input_id;
        //Провера да ли је играч елиминисан
        if player.respawn_timer > 0.0 {
            player.mouse_angle = input.mouse_angle;
            return;
        }

        let mut reset_reloads: bool = false;
        let mut player_position_x: f32 = 0.0;
        let mut player_position_y: f32 = 0.0;
        if let Some(rb) = self.rigid_body_set.get_mut(player.body_handle) {
            let speed = 10.0;
            let mut x_vel = 0.0;
            player_position_x = rb.position().translation.x;
            player_position_y = rb.position().translation.y;

            player.horizontal_velocity = 0.0;
            if input.move_left {
                player.horizontal_velocity -= speed;
            }
        }
        ...остатак методе
    }
}
```

Листинг 4: Део методе за валидацију и обраду корисничког уноса.

5.2.3 Детекција десинхронизације и усклађивање (клијент)

Процес усклађивања започиње обрадом примљеног пакета и изменом података о тренутном оружју играча као што су муниција и репетирање оружја (приказано на листингу 5), након чега се бришу застареле команде из бафера (видети листинг 6). Коначно, на основу разлике између предвиђене и ауторитативне позиције се примењује механизам корекције који је приказан на листингу 7. Ако је разлика у позицији велика, клијент додатно понавља све акције које су извршене у периоду између два пакета (овај поступак омогућава да клијент не изгуби уносе док је чекао одговор сервера).

```

func handle_server_response(player_snapshot: Dictionary):
    target_position = Vector2(player_snapshot["position"][0] * METER_TO_PIXEL,
player_snapshot["position"][1] * METER_TO_PIXEL) #Позиција играча одређена сервером.
    var last_processed_id = player_snapshot["last_processed_input_id"] #Последњи унос који је
сервер обрадио.

    if not throwable_map.has(player_snapshot["gun"]):
        weapons[weapon_index].update_from_server(player_snapshot)
        if weapons_names_list[weapon_index] != player_snapshot["gun"]:
            weapons[weapon_index].reload_sound.stop()
    else:
        throwables[throwable_map[player_snapshot["gun"]]].set_snapshot(player_snapshot)

```

Листинг 5: Очитавање позиције са сервера и ажурирање оружја.

```

while len(inputs_list) > 0 and inputs_list[0].input_id <= last_processed_id:
    inputs_list.remove_at(0)

```

Листинг 6: Брисање застарелих команди из бафера.

```

var checking_state = null
var match_index: int = -1
for i in range(len(state_history)):
    if state_history[i]["id"] == last_processed_id:
        checking_state = state_history[i]
        match_index = i
        break

if checking_state != null:
    var error_vec = target_position - checking_state["global_position"]

    var distance = error_vec.length()
    #ТЕШКА КОРЕКЦИЈА
    if distance > 100.0:
        global_position = target_position
        for input_item in inputs_list:
            apply_movement_correction(input_item, PHYSICS_DELTA)

    #ЛАГАНА КОРЕКЦИЈА
else:
    var old_pos = global_position
    global_position = target_position
    var new_predicted_pos = global_position
    var offset = old_pos - new_predicted_pos
    visuals.global_position += offset

```

Листинг 7: Кориговање позиције клијента.

5.3 Имплементација кретања играча - *Command* и *State* шаблон

Како би серверско усклађивање имало смисла и клијентска предикција била могућа, неопходно је да клијент и сервер рачунају кретање на исти начин. *Godot Game Engine* има уграђену методу `move_and_slide()` која рачуна кретање помоћу његове интерне физике, а *Rapier* библиотека у *Rust*-у помоћу своје. Стога, било је потребно ручно написати логику кретања играча, како на клијенту, тако и на серверу. Олакшана околност је што и *Godot Game Engine* и *Rapier* библиотека пружају могућност аутоматског рачунања колизија објеката на основу прослеђених података, чиме није било потребе за додатном имплементацијом рачунања.

5.3.1 Имплементација на клијентској страни

5.3.1.1 *Command* шаблон

За потребе игрице, *Command* шаблон је искоришћен за енкапсулацију акције кретања (да ли је играч у посматраном фрејму притиснуо тастер за кретање лево, десно или скок), приказано у листингу 8.

```
extends Node
class_name PlayerMoveCommand

var player: MyPlayer
var move_left: bool
var move_right: bool
var jump: bool
var input_id: int

func _init(player: MyPlayer, input_id: int) -> void:
    self.player = player
    self.move_left = false
    self.move_right = false
    self.jump = false
    self.input_id = input_id

func execute(delta: float):
    self.player.player_state.update(delta, self.player, self)
```

Листинг 8: Имплементација *Command* класе.

Једина метода коју *PlayerMoveCommand* класа садржи је `execute()` која се позива у раније наведеној `apply_movement_step` методи, у секцији 5.2.1. Једина улога ове методе је да позове `update()` методу која се налази у *PlayerState* класи преко референце на играча.

5.3.1.2 *State* шаблон

У зависности од извршених акција и стања игре, играч у посматраном тренутку може да пуца, трчи, скаче или чека да оживи након што је био елиминисан. Како би се избегле сталне *if* провере у `physics_process()` методи која се позива сваког фрејма, што, ако их има пуно, може да доведе до пада у перформансама, имплементиран је *State* шаблон представљен *PlayerState* апстрактном класом (видети листинг 9). `Enter()` и `update()` методе морају бити редефинисане у конкретним имплементацијама класа, док су `apply_common_physics()` и `handle_inputs()` заједничке методе за сва стања. Постојеће класе стања су: *IdleState*, *RunningState*, *JumpingState* и *DeadState*, а њихове имплементације `enter()` методе су приказане у листинзима 10, 11, 12 и 13. Играч садржи референцу на стање, а када је потребно да играч пређе у ново стање, потребно је позвати методу `change_player_state()` (приказано на листингу 14).

Употреба *State* шаблона омогућава организованији и прегледнији код, боље перформансе и лакше дебаго-

бање, јер је потребно фокусирати се само на оно стање које је изазвало проблем. Листинг 15 приказује псеудокод `execute()` методе из листинга 8 без имплементације шаблона.

```
@abstract class_name PlayerState
extends Node

@abstract
func enter(player: MyPlayer);

@abstract
func update(delta: float, player: MyPlayer, command: PlayerMoveCommand);

func apply_common_physics(delta: float, player: MyPlayer, command: PlayerMoveCommand):
    var direction = 0
    if not player.is_dead:
        if command.move_left: direction -= 1
        if command.move_right: direction += 1

        if direction > 0 and not player.can_move_right: direction = 0
        elif direction < 0 and not player.can_move_left: direction = 0
    else:
        direction = 0
    var motion_x = direction * player.SERVER_SPEED * player.METER_TO_PIXEL
    player.move_and_collide(Vector2(motion_x * delta, 0))

    var predicted_v_velocity = player.vertical_velocity + player.GRAVITY * delta
    if predicted_v_velocity > 12.0:
        predicted_v_velocity = 12.0

    var motion_y = predicted_v_velocity * delta * player.METER_TO_PIXEL
    var collision = player.move_and_collide(Vector2(0, motion_y))

    player.is_on_ground = false

    if collision:
        var normal = collision.get_normal()
        if normal.y < -0.5:
            player.is_on_ground = true
            player.vertical_velocity = 0.0
        elif normal.y > 0.5:
            player.vertical_velocity = 0.0
        else:
            player.vertical_velocity = predicted_v_velocity

    return direction

func handle_inputs(player: MyPlayer):
    if Network.INPUT_DATA["gun"] == "m4a1_rifle":
        Network.INPUT_DATA["shoot"] = Input.is_action_pressed("shoot")
    else:
        Network.INPUT_DATA["shoot"] = Input.is_action_just_pressed("shoot")
#ОСТАТАК МЕТОДЕ
```

Листинг 9: Базна *PlayerState* класа

```
class_name IdleState extends PlayerState

func enter(player: MyPlayer):
    player.vertical_velocity = 0
    player.is_on_ground = true
    player.walking_sprite.visible = false
    player.idle_sprite.visible = true
    player.dying_sprite.visible = false
    player.animation_player.play("idle_animation")
```

Листинг 10: Имплементација *enter()* методе **IdleState** класе

```
class_name RunningState extends PlayerState

func enter(player: MyPlayer):
    player.walking_sprite.visible = true
    player.idle_sprite.visible = false
    player.dying_sprite.visible = false
    player.animation_player.play("walking_animation")
```

Листинг 11: Имплементација *enter()* методе **RunningState** класе

```
class_name JumpingState extends PlayerState

func enter(player: MyPlayer):
    player.is_on_ground = false
    player.walking_sprite.visible = true
    player.idle_sprite.visible = false
    player.dying_sprite.visible = false
    player.animation_player.play("walking_animation")
```

Листинг 12: Имплементација *enter()* методе **JumpingState** класе

```
class_name DeadState extends PlayerState

func enter(player: MyPlayer):
    player.is_on_ground = true
    player.walking_sprite.visible = false
    player.idle_sprite.visible = false
    player.dying_sprite.visible = true
    player.health_amount.visible = false

    player.is_dead = true
    player.can_move_left = false
    player.can_move_right = false
    player.animation_player.play("dying_animation")
```

Листинг 13: Имплементација *enter()* методе **DeadState** класе

```
func change_state(state: PlayerState):
    if self.player_state:
        self.player_state.queue_free()
    self.player_state = state
    self.player_state.enter(self)
```

Листинг 14: Имплементација *change_state()* методе **MyPlayer** класе

```
#Пуно if провера сваки фрејм
func execute(delta: float):
    if играч_HP <= 0.0:
        return
    if играч_није_на_земљи:
        код за логику скакања...
    else:
        if играч_се_креће:
            код за логику кретања...
        else:
            код за логику стајања...
```

Листинг 15: Псеудокод *execute()* методе ***PlayerMoveCommand*** класе без имплементације ***State*** шаблона.

5.3.2 Имплементација на серверској страни

Како је споменуто, било је потребно дефинисати и кретање играча на серверској страни. Део имплементације *handle_movement()* методе у *Player* структури је приказан у листингу 16. Ако се упореди са методом *apply_common_physics()* из листинга 9, може се видети да се кретање рачуна на исти начин. Вредност константе гравитације и максимална дозвољена вертикална брзина су исте, као и начин провере ли је играч на земљи или ударио у терен изнад.

Постоје мале разлике које увек доводе до малог одступања између позиција на клијентској и серверској страни, без обзира како се кретање рачуна:

- *Godot Game Engine* као мерну јединицу користи пиксел, док *Rapier* библиотека користи метар. Одлучено је да сваки метар на серверу буде представљен као 32 пиксела на клијенту дефинисањем *METER_TO_PIXEL* константе. Стога се на клијенту, пре рачуна колизија, хоризонтална и вертикална брзина множе са дефинисаном константом, што доводи до минималног одступања приликом заокруживања вредности.
- Чак и уз примену *tick-rate* синхронизације, где је одређено да се физика рачуна 60 пута у секунди, због начина на који *Godot Game Engine* и *Rapier* заокружују децималне вредности, кроз велики број итерација може доћи до благог одступања у срачунатим позицијама (до неколико пиксела кроз одређени период).

```

pub fn handle_movement(
    &mut self,
    custom_gravity: Vec2,
    delta: f32,
    rigid_body_set: &mut RigidBodySet,
    collider_set: &mut ColliderSet,
    broad_phase: &BroadPhaseBvh,
    narrow_phase: &NarrowPhase,
    char_controller: KinematicCharacterController,
) {
    let mut translation_to_apply = None;
    {
        if self.is_on_ground && self.vertical_velocity >= 0.0 {
            self.vertical_velocity = 0.0;
        }
        self.vertical_velocity += custom_gravity.y * delta;
        if self.vertical_velocity > 12.0 {
            self.vertical_velocity = 12.0;
        }
        if let Some(rb) = rigid_body_set.get(self.body_handle) {
            //...КОД ЗА ФИЛТРИРАЊЕ КОЛИЗИЈЕ НИЈЕ ПРИКАЗАН
            let pos_after_x = rb.position().translation + result_x.translation;
            let mut temp_pose = Pose::new(Vec2::new(pos_after_x.x, pos_after_x.y), 0.0);
            let vertical = vec2(0.0, self.vertical_velocity * delta);
            self.is_on_ground = false;
            let mut hit_ceiling = false;
            //Провера да ли је играч на земљи, у ваздуху или је ударио терен изнад
            let result_y = char_controller.move_shape(
                delta,
                &queries,
                collider.shape(),
                &temp_pose,
                vertical,
                |collision| {
                    let normal = collision.hit.normal;
                    if normal.y < -0.5 {
                        if self.vertical_velocity >= 0.0 {
                            self.is_on_ground = true;
                            self.vertical_velocity = 0.0;
                        }
                    }
                    if normal.y > 0.5 {
                        if self.vertical_velocity < 0.0 {
                            hit_ceiling = true;
                        }
                    }
                },
            );
            let mut final_translation = result_x.translation + result_y.translation;
            translation_to_apply = Some(final_translation);
        }
    }
}

```

Листинг 16: Део имплементације *handle_movement()* методе задужене за рачунање позиције играча.

5.4 Комуникација клијента и сервера

5.4.1 Протоколи

Комуникација између клијента и сервера је подељена на три различита протокола:

- **UDP протокол:** Користи се искључиво док траје игра стартованог *lobby*-ја. Како играчи константо шаљу своје акције ка серверу и није дозвољено чекање потврде сервера да ли је пакет успешно примљен, што би изазвало латенцију, користи се UDP протокол. Такође, након што израчуна ново стање игре, сервер шаље путем истог протокола пакете ка свим повезаним клијентима у партији. Ако неки пакет не стигне, серверским усклађивањем следећег добијеног пакета ће играч бити позициониран на неопходну позицију.
- **WebSocket:** Конекција се успоставља оног тренутка, када играч приступи жељеном *lobby*-ју. Користи се за акције које се често извршавају, а неопходно је да клијент и сервер добију одговор. У акције спадају:
 - мењање мапе,
 - мењање неопходног броја елиминације за победу (ако је FFA режим) или HP куле,
 - мењање изгледа играча,
 - четовање и
 - обавештавање када је играч елиминасан.

Када играч изађе из партије или *lobby*-ја, конекција се прекида.

- **HTTP протокол (REST API):** Користи се за акције ван саме партије које се извршавају повремено, а за које је потребно потврда извршења, као што су:
 - пријава,
 - одјава,
 - регистрација,
 - креирање, улазак и излазак из *lobby*-ја и
 - добијање информација о приступљеном *lobby*-ју.

На листингу [17](#) се могу видети сви енуми који се користе за размену података између клијента и сервера. **ServerMessage** представља одговор сервера ка клијенту, док је **ClientMessage** структура коју клијент шаље серверу.

```

#[derive(Serialize, Deserialize)]
pub enum ServerMessage {
    Init(u32), //0
    Snapshot(GameState), //1
    Pong(u64), //2
    GameEnd(GameEnd), //3
    LobbiesList(LobbiesInfo), //4
    CreatedLobbyResponse(u32, u32), //5
    GameStarted(bool), //6
    LobbyInfo(LobbyRoomInfo), //7
    PlayerDisconnected(u32, u32), //8 player_id lobby_host_id
    PlayerChangedSkin(u32, u8), //9 player_id, player_skin_index(0-GREEN,1-BLUE,2...)
    PlayerChangedReadyState(u32), //10 player_id
    TowerMaxHPChanged(u32), //11 tower_max_hp
    PlayerMessage(u32, String), //12 player_id, message
    PlayerConnected(u32, String), //13 player_id, player_nickname
    PlayerKilled(u32, u32, GunEnum), //14 killer_id, victim_id, gun_index (0-pistol, 1-m4a1)
    KillsToWinChanged(u8), //15 kill_amount
    AuthenticationResponse(u32, String, String), //16 player_id, nickname, token
    MapChanged(u8), //17 map_index
    StartedLobbyJoinResponse(u8), //18 map_index
    TowerCreated(TowerSnapshot) //19
}

#[derive(Deserialize, Debug)]
pub enum ClientMessage {
    Input(ClientInput), // 0
    PingCheck(PingInput), // 1
    LobbyCreate(CreateLobbyRequest), //2
    LobbyJoin(JoinRequest), //3,
    LobbyStart(u32), //4 lobby_id
    PlayerReady(u32), //5 lobby_id
    GetLobbyInfo(u32), //6 lobby_id
    ChangeTowerMaxHP(u32, u32), //7 lobby_id, tower_max_hp
    ChangePlayerBodySkin(u32, u8), //8 lobby_id, skin_index (0-GREEN,1-BLUE,2-RED)
    LobbyLeave(u32), //9 lobby_id
    PlayerMessage(u32, String), //10 lobby_id, message
    ChangeKillsToWin(u32, u8), //11 lobby_id, kill_amount
    JoinStartedLobby(u32), //12 lobby_id
    RegistrationData(String, String), //13 nickname, password
    LoginData(String, String, u8), //14 nickname, password, //project version
    ChangeMap(u8), //15, map_index
}

```

Листинг 17: Приказ енума који се користе за размену података између клијента и сервера.

5.4.2 JWT

JWT представља додатан слој заштите и аутентификације како би сервер могао безбедно да идентификује корисника без потребе да чува податке о сесији у својој меморији или да за сваки захтев проверава базу података.

Приликом пријаве, сервер генерише токен приказаном у листингу 18 и смешта га као параметар **AuthenticationResponse** варијанте **ServerMessage** енума, а клијент потом чува токен. Клијент складишти токен у променљивој, шаље га у *Authorization* заглављу у сваком наредном захтеву ка серверу и брише га када се одјави.

```

static JWT_SECRET: Lazy<Vec<u8>> = Lazy::new(|| {
    std::env::var("JWT_SECRET")
        .expect("JWT_SECRET nije postavljen!")
        .into_bytes()
});

pub struct JWTHandler;

impl JWTHandler {
    pub fn create_jwt(player_id: u32, player_nickname: String) -> String {
        let now = Utc::now();
        let expire = now + Duration::hours(24);

        let claims = Claims {
            id: player_id,
            nickname: player_nickname,
            iat: now.timestamp() as usize,
            exp: expire.timestamp() as usize,
        };

        encode(
            &Header::default(),
            &claims,
            &EncodingKey::from_secret(&JWT_SECRET),
        )
        .unwrap()
    }

    pub fn validate_jwt(token: &str) -> Result<Claims, jsonwebtoken::errors::Error> {
        decode::<Claims>(
            token,
            &DecodingKey::from_secret(&JWT_SECRET),
            &Validation::new(Algorithm::HS256),
        )
        .map(|data| data.claims)
    }
}

```

Листинг 18: Имплементација `create_jwt()` и `validate_jwt()` метода

5.4.3 *Network* и *MyHttpHandler* синглтони (клијент)

Синглтон *Network* (приказан на листингу 19) је искључиво задужен за успостављање, одржавање и управљање мрежним конекцијама које захтевају брз проток података, првенствено путем UDP протокола и *WebSocket*-а.

UDP комуникација (*PacketPeerUDP*): Како је приказано у методи `connect_to_socket()`, клијент врши резолуцију хоста преко `IP.resolve_hostname()` како би динамички претворио доменско име или локалну адресу у валидну IP адресу. Коришћењем класе ***PacketPeerUDP*** омогућено је слање сирових бајтова (*PackedByteArray*) уз минимално кашњење, што је критично за игре са брзом синхронизацијом позиција играча и стања физичке симулације. Метода `send_data()` врши безбедну проверу активне конекције пре саме серијализације и слања пакета ка серверу на дефинисаном порту (у овом случају 9000).

WebSocket комуникација: Поред UDP-а, синглтон интегриса и ***WebSocketPeer*** механизам (дефинисан кроз променљиве попут: `websocket`, `is_connected_to_websocket` и праћење губитка везе `is_conenction_with_websocket_lost`). Ово омогућава поуздан, двосмерни комуникацијски канал преко TCP-а за догађаје који захтевају гарантовану испоруку порука, а који нису временски критични као сама физичка симулација.

Са друге стране, ***MyHttpHandler*** синглтон (приказан на листингу 20) се ослања на REST API комуникацију. Архитектура овог синглтона се заснива на асинхроном моделу захтев-одговор (*Request-Response*), где свака акциона метода има свој пратећи повратни позив (енгл. *callback* методу) која се окида по завршетку HTTP захтева (`register()` - `on_register_completed()`, `login()` - `on_login_completed()`). Такође, иако је ***Network*** задужен за успостављање конекције, ***MyHttpHandler*** синглтон је задужен и за слање података ка серверу путем *WebSocket*-а.

Аутентификација и регистрација: Методе `register()` и `login()` омогућавају слање креденцијала корисника ка серверу, док одговарајуће функције `on_register_completed()` и `on_login_completed()` прихватају повратне параметре (`result`, `response_code`, `headers`, `body`, `http_node`) и обрађују статус код сервера (нпр. издавање и чување JWT токена).

Управљање *lobby*-јима: Методе попут `get_all_lobbies()` и `create_lobby_binary()` омогућавају преглед доступних соба и креирање нових партија са специфичним параметрима (максималан број играча, шифра *lobby*-ја, број режима игре).


```

extends Node2D
var is_local: bool = false
const VERSION: int = 1
###CONNECTION
var socket := PacketPeerUDP.new()
var server_address = null
var server_port := 9000
var is_connected_to_udp_socket: bool = false
var websocket := WebSocketPeer.new()
var websocket_address = null
var is_connected_to_websocket: bool = false
var is_conenction_with_websocket_lost: bool = false
...остатак променљивих

#Метода за отварање UDP сокета
func connect_to_socket():
    var ip = IP.resolve_hostname(server_address)
    if ip == "" or not ip.is_valid_ip_address():
        return

    var err = socket.set_dest_address(ip, server_port)
    if err != OK:
        return

    is_connected_to_udp_socket = true
#Метода за слање података серверу
func send_data(data: PackedByteArray):
    if is_connected_to_udp_socket:
        socket.put_packet(data)

...остале методе

```

Листинг 19: Део *Network* синглтона

```

extends Node
func register(nickname: String, password: String)
func _on_register_completed(result, response_code, headers, body, http_node)
func login(nickname: String, password: String)
func _on_login_completed(result, response_code, headers, body, http_node)
func get_all_lobbies()
func _on_get_all_lobbies_completed(result, response_code, headers, body, http_node)
func create_lobby_binary(max_players: int, password: String, game_mode_number: int = 0)
func _on_create_completed(result, response_code, headers, body, http_node)
...остале методе

```

Листинг 20: Део *MyHttpHandler* синглтона

5.4.4 Оптимизација величине пакета

У циљу смањења мрежног оптерећења (*network overhead*) и кашњења, комуникација између клијента и сервера у овој апликацији је реализована коришћењем бинарне серијализације података уместо текстуалних формата као што је JSON. Иако је JSON читљив и једноставан за имплементацију, он је за потребе мрежне синхронизације у реалном времену неефикасан из следећих разлога:

- У JSON-у, сваки податак је идентификован кључем. На пример, запис {"position_x": 140.0} троши бајтове и на назив кључа који се понавља у сваком пакету. Бинарни запис преноси само вредности, што драстично смањује величину пакета.
- Додатно, одређено време се троши на десеријализацију самог формата, што код бинарне серијализације није случај.

Листинг 21 приказује методу `convert_input_data_to_byte_array()` **Network** синглтона, која је задужена за припрему података за слање ка серверу. **INPUT_DATA** представља речник у којем се налазе неопходни подаци које клијент шаље ка серверу како би сервер могао успешно да изврши симулацију.

Поређења ради, употребом JSON формата, сваког фрејма клијент шаље пакете величине око 180 бајтова, док се употребом бинарне серијализације величина пакета смањује на 37 бајтова:

- `message_type` (u32): 4 бајта,
- `my_id` (u32): 4 бајта,
- `input_id` (u32): 4 бајта,
- `boolean` поља (u8 по пољу): 4 бајта,
- `mouse_angle` (float / 32-bit): 4 бајта,
- `cmd_id` (u32): 4 бајта,
- `gun_id` (u32): 4 бајта,
- провера за позицију метка (u8 за 0 или 1): 1 бајт,
- `bullet_spawn_position` (2 пута по 4 бајта за float): 8 бајтова

Како би се додатно уштедело на величини пакета, у неким ситуацијама је искоришћено и битовско померање. Уместо да се подаци о томе да ли је играч на земљи, да ли репетира оружје или гледа надесно прослеђују као три засебна бајта, шаље се 1 бајт. Коришћењем битовског "и" са одређеном маском и поређењем добијене вредности са нулом, добијају се *boolean* вредности. Листинг 22 приказује читање бинарног формата пакета добијеног од сервера.

Важно је напоменути да је редослед којим се подаци читају и пакују битан. Нарушавање редоследа само једног поља може довести до десинхронизације и погрешног тумачења података.

```

    INPUT_DATA = {
        "player_id": my_id,
        "input_id": 0,
        "move_left": false,
        "move_right": false,
        "jump": false,
        "shoot": false,
        "mouse_angle": 0.0,
        "command": "JOIN",
        "gun": "pistol",
        "bullet_spawn_position": null
    }

    func convert_input_data_to_byte_array():
var buffer = StreamPeerBuffer.new()

buffer.put_u32(0) # ClientMessage::Input
buffer.put_u32(my_id)
buffer.put_u32(INPUT_DATA["input_id"])

buffer.put_u8(1 if INPUT_DATA["move_left"] else 0)
buffer.put_u8(1 if INPUT_DATA["move_right"] else 0)
buffer.put_u8(1 if INPUT_DATA["jump"] else 0)
buffer.put_u8(1 if INPUT_DATA["shoot"] else 0)

buffer.put_float(INPUT_DATA["mouse_angle"])

var cmd_id = Command.get(INPUT_DATA["command"], 0)
buffer.put_u32(cmd_id)

var gun_id = Gun.get(INPUT_DATA["gun"].to_upper(), 0)
buffer.put_u32(gun_id)

if INPUT_DATA["bullet_spawn_position"] == null:
    buffer.put_u8(0)
else:
    buffer.put_u8(1)
    buffer.put_float(INPUT_DATA["bullet_spawn_position"][0])
    buffer.put_float(INPUT_DATA["bullet_spawn_position"][1])

return buffer.data_array

```

Листинг 21: Припрема података за слање корисничких акција ка серверу.

```
func create_players_snapshot(buffer: StreamPeerBuffer):  
    var snapshot: Dictionary = {}  
  
    snapshot["id"] = buffer.get_u32()  
  
    var name_length = buffer.get_u64()  
    snapshot["nickname"] = buffer.get_utf8_string(name_length)  
  
    var pos_x = buffer.get_float()  
    var pos_y = buffer.get_float()  
    snapshot["position"] = Vector2(pos_x, pos_y)  
  
    snapshot["hp"] = buffer.get_32()  
  
    #const FLAG_FACING_RIGHT = 1 (00000001)  
    #const FLAG_IS_ON_GROUND = 2 (00000010)  
    #const FLAG_IS_RELOADING = 4 (00000100)  
    var flags_byte: int = buffer.get_u8()  
    var facing_right = (flags_byte & FLAG_FACING_RIGHT) != 0  
    var is_on_ground = (flags_byte & FLAG_IS_ON_GROUND) != 0  
    var is_reloading = (flags_byte & FLAG_IS_RELOADING) != 0  
  
    snapshot["facing_right"] = facing_right  
    snapshot["is_on_ground"] = is_on_ground  
    snapshot["is_reloading"] = is_reloading  
  
    snapshot["respawn_timer"] = buffer.get_float()  
    snapshot["last_processed_input_id"] = buffer.get_u32()  
    snapshot["mouse_angle"] = buffer.get_float()  
  
    var gun_id = buffer.get_u32()  
    if gun_id == 0:  
        snapshot["gun"] = "pistol"  
    elif gun_id == 1:  
        snapshot["gun"] = "m4a1_rifle"  
    elif gun_id == 2:  
        snapshot["gun"] = "grenade"  
  
    snapshot["current_ammo"] = buffer.get_16()  
    snapshot["player_skin"] = buffer.get_u8()  
    snapshot["player_score"] = buffer.get_u8()  
    snapshot["velocity_x"] = buffer.get_u32()  
    snapshot["velocity_y"] = buffer.get_u32()  
  
    return snapshot
```

Листинг 22: Читање бинарног формата пакета добијеног од сервера.

У оквиру овог дипломског рада успешно је дизајнирана, развијена и имплементирана игра *Commander's Defense* заснована на клијент-сервер архитектури. Пројекат обухвата потпуну интеграцију клијентске стране реализоване у *Godot Game Engine*-у и високих перформанси серверске логике писане у *Rust* програмском језику, чиме је обезбеђена стабилност, безбедност и синхронизација система у реалном времену. Кроз рад су успешно обрађени и имплементирани следећи кључни сегменти:

- Анализирани су концепти *online multiplayer* система, серверског усклађивања, *Command* шаблона, клијентске предикције и *tick-rate* синхронизације.
- Имплементиран је *State* шаблон на клијентској страни ради боље организације и читљивости кода, као и повећање перформанси система.
- Употреба хибридне мрежне комуникације. Остварена је оптимална комбинација UDP протокола за временски критичну синхронизацију кретања и физике (ниско кашњење), *WebSocket* конекција за поуздан пренос догађаја, као и REST API руковаоца (*MyHttpHandler*) за аутентификацију и управљање *lobby*-јима.
- Уместо тешких текстуалних формата (попут JSON-а), целокупна мрежна комуникација заснована је на ефикасној бинарној серијализацији, чиме је величина пакета драстично смањена, а пропусни опсег (енгл. *bandwidth*) оптимизован.

Главна предност изабране клијент-сервер архитектуре јесте апсолутна валидност података и сигурност коју централизовани сервер пружа. Пошто сервер води главну реч и проверава физику и улазе, спречене су разне врсте варања које су карактеристичне за децентрализоване системе.

Међутим, основна мана овог приступа огледа се у томе што клијент-сервер архитектура у контексту специфичних захтева игре може представљати веће оптерећење у поређењу са P2P архитектуром. Централни сервер неизбежно постаје уско грло (енгл. *bottleneck*) система, јер сваки клијент мора да шаље податке ка серверу и чека његов одговор, што директно утиче на скалабилност система и захтева веће серверске ресурсе како се број истовремених играча повећава. Стога, у будућим фазама развоја систем би могао прећи на хибридни модел где би се сервер користио само за успостављање везе и вођење евиденције о *lobby*-јима, док би међусобна комуникација играча ишла директно између клијената. Тиме би се смањило оптерећење сервера.

Списак слика

Слика 1	<i>Peer-to-Peer</i> архитектура.	4
Слика 2	клијент-сервер модел.	4
Слика 3	хибридни модел.	5
Слика 4	<i>Commando Assault</i> видео игра.	9
Слика 5	<i>Strike Force Heroes</i> видео игра.	9
Слика 6	Почетни мени.	10
Слика 7	Мени за регистрацију.	10
Слика 8	Главни мени.	11
Слика 9	Мени за креирање <i>lobby</i> -ја.	11
Слика 10	Пример креираног <i>lobby</i> -ја - FFA режим.	12
Слика 11	Пример креираног <i>lobby</i> -ја - кула режим.	12
Слика 12	Изглед UI-ја игре покренуте на рачунару.	13
Слика 13	Изглед UI-ја игре покренуте на мобилном телефону.	13
Слика 14	Изглед паузирајућег менија.	14
Слика 15	Изглед <i>scoreboard</i> -а са два играча у партији - FFA режим.	14
Слика 16	Изглед <i>scoreboard</i> -а са два играча у партији - FFA режим.	15
Слика 17	Изглед <i>scoreboard</i> -а са два играча у партији.	15
Слика 18	Изглед <i>scoreboard</i> -а са два играча у партији.	16
Слика 19	Изглед табеле <i>players</i> у бази података.	17
Слика 20	Дијаграм секвенци који приказује ток синхронизације стања.	18

Списак листинга

Листинг 1	Имплементација позива	18
Листинг 2	<i>apply_movement_step</i> метода	19
Листинг 3	Слање података ка серверу и чување команди у бафер.	19
Листинг 4	Део методе за валидацију и обраду корисничког уноса.	20
Листинг 5	Очитавање позиције са сервера и ажурирање оружја.	21
Листинг 6	Брисање застарелих команди из бафера.	21
Листинг 7	Кориговање позиције клијента.	21
Листинг 8	Имплементација <i>Command</i> класе.	22
Листинг 9	Базна <i>PlayerState</i> класа	23
Листинг 10	Имплементација <i>enter()</i> методе <i>IdleState</i> класе	24
Листинг 11	Имплементација <i>enter()</i> методе <i>RunningState</i> класе	24
Листинг 12	Имплементација <i>enter()</i> методе <i>JumpingState</i> класе	24
Листинг 13	Имплементација <i>enter()</i> методе <i>DeadState</i> класе	24
Листинг 14	Имплементација <i>change_state()</i> методе <i>MyPlayer</i> класе	24
Листинг 15	Псеудокод <i>execute()</i> методе <i>PlayerMoveCommand</i> класе без имплементације <i>State</i> шаблона.	25
Листинг 16	Део имплементације <i>handle_movement()</i> методе задужене за рачунање позиције играча. ...	26
Листинг 17	Приказ енума који се користе за размену података између клијента и сервера.	28
Листинг 18	Имплементација <i>create_jwt()</i> и <i>validate_jwt()</i> метода	29
Листинг 19	Део <i>Network</i> синглтона	31
Листинг 20	Део <i>MyHttpHandler</i> синглтона	31
Листинг 21	Припрема података за слање корисничких акција ка серверу.	33
Листинг 22	Читање бинарног формата пакета добијеног од сервера.	34

Биографија

Овде написати своју кратку биографију.

Литература

- [1] J. Ryška, „Development and forecast of esports industry“, *Prague Economic Papers*, том 31, изд. 2, стр. 215–235, 2022.
- [2] J. D. Pellegrino и С. Dovrolis, „Bandwidth requirement and state consistency in three multiplayer game architectures“, у *Proceedings of the 2nd workshop on Network and system support for games*, New York, NY, USA: ACM, 2003, стр. 151–159.
- [3] H. Viitanen, „Deterministic and Synchronous Computation Between Client and Server in Mobile Games“, Master's Thesis, 2025.
- [4] S. D. Webb и S. Soh, „Cheating in networked computer games: a review“, у *Proceedings of the 2nd International Conference on Digital Interactive Media in Entertainment and Arts*, у DIMEA '07. ACM, 2007, стр. 105–112.
- [5] W. Bugden и A. Alahmar, „Rust: The programming language for safety and performance“, *arXiv preprint arXiv:2206.05503*, 2022.

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `pyflies` недостаје поље `urldate`